3(a): Compare CUBIC, Reno, and BBR

We compared CUBIC, Reno, and BBR at three destinations. Each algorithm had three 60-second runs per destination, sampled once per second: 27 matched runs. We selected each algorithm before connecting the data socket and recorded the name read back from it. The host used Linux 6.18.33.2-microsoft-standard-WSL2, Python 3.14.8, matplotlib 3.11.2, and pydantic 2.13.5.

We shuffled destinations with seed 422 and rotated the algorithm order. Dallas used BBR/CUBIC/Reno, CUBIC/Reno/BBR, then Reno/BBR/CUBIC. The other two destinations started with the second and third orders. Tests ran sequentially with a two-second pause. Dallas was the first complete group and became the representative. Five incomplete groups were excluded from comparisons; their successful runs and failures remain saved.

Goodput is acknowledged bytes divided by actual elapsed time, converted to Mbit/s. Each algorithm mean averages its three run rates; ± shows their sample standard deviation. RTT is the average sampled smoothed RTT. Retransmissions indicate possible loss, not an exact loss probability. Per-test tables also give minimum, median, mean, and P95 goodput; P95 uses linear interpolation.

Dest.	Algorithm	Goodput (Mbit/s)	RTT (ms)	Retrans.
D1	cubic	273.96 ± 5.63	68.73	224
D1	reno	281.30 ± 2.76	73.65	444
D1	bbr	269.79 ± 11.69	64.53	196
D2	cubic	118.08 ± 4.16	181.90	23
D2	reno	119.07 ± 0.89	181.67	41
D2	bbr	106.59 ± 17.37	177.39	398
D3	cubic	163.12 ± 17.51	132.58	120
D3	reno	163.52 ± 11.80	136.69	11
D3	bbr	147.47 ± 6.02	150.60	541

D1: dal.speedtest.clouvider.net:5200
D2: speed.netfiber.net.il:5201
D3: 96.45.41.167:30001

Reproduce: python main.py analyze --manifest results/algorithm-comparisons/q3-test/manifest.json

Raw data and full min/median/mean/P95 tables: results/algorithm-comparisons/q3-test/

Repository
Data socket selection
Experiment runner
Statistics and plots

3(a): Observations and limitations

Congestion window (cwnd) limits how much data the sender may have outstanding. RTT measures the delay before acknowledgements return. Retransmissions show data being sent again. Together, these help explain why a larger window may not produce higher goodput.

Dallas: Reno averaged 281.30 Mbit/s, CUBIC 273.96, and BBR 269.79. BBR had the lowest mean RTT, 64.53 ms; Reno had the highest, 73.65 ms. Reno's median cwnd grew from 1,982 to 7,854 segments across runs, but goodput stayed near 279–284 Mbit/s. The extra window did not produce a similar increase in delivery.

Netfiber: Reno and CUBIC were close at 119.07 and 118.08 Mbit/s, with mean RTTs near 182 ms. BBR had slightly lower RTT but averaged 106.59 Mbit/s. Its third run fell to 86.57 Mbit/s and recorded 396 retransmissions. RTT remained near baseline during much of the slowdown, so RTT alone would not describe this problem.

IP destination: Reno and CUBIC were almost tied at 163.52 and 163.12 Mbit/s. Their run standard deviations were 11.80 and 17.51 Mbit/s, much larger than the difference between their means. BBR averaged 147.47 Mbit/s. Around 50 seconds into its first run, one interval had zero goodput and RTT reached 1,559.64 ms. The transfer completed, and we kept this stall in the statistics.

Reno had the highest mean goodput on all three paths. BBR had lower mean RTT on Dallas and Netfiber. We did not observe a CUBIC or BBR throughput win. Reno grows its window additively [1], CUBIC uses a cubic growth rule [2], and BBR estimates delivery rate and baseline RTT [3]. These explain their intended behavior, but do not establish the cause of each measured difference.

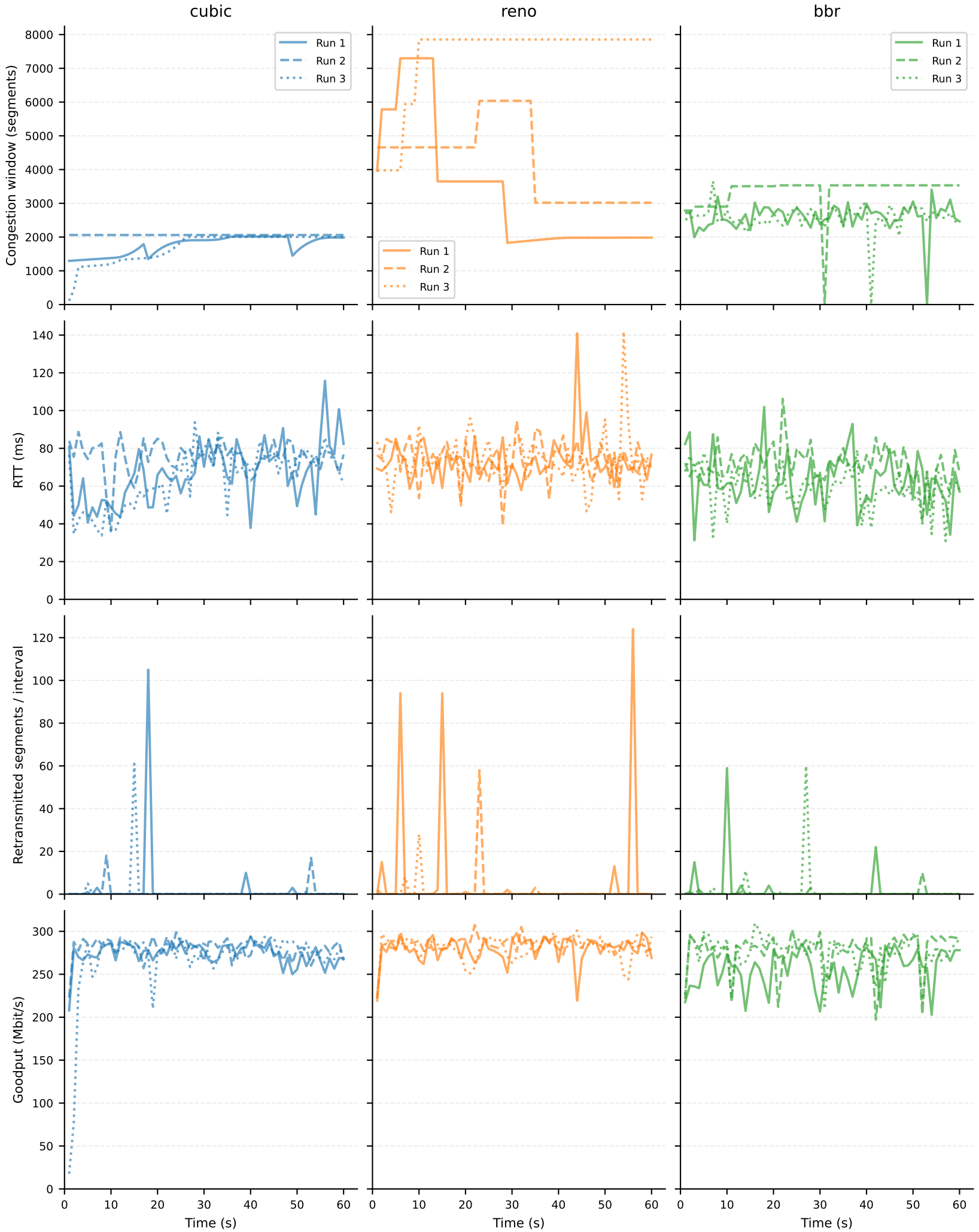
Bandwidth-delay product (BDP) is bandwidth × propagation RTT: roughly the data needed in flight to fill a path. We used achieved goodput × minimum RTT as an approximate comparison. Dallas's third Reno run allowed about 11.37 MB through cwnd, versus a 1.09 MB rate × RTT estimate. A large window alone therefore did not explain throughput. This estimate does not independently measure path capacity; receiver or server limits may also matter.

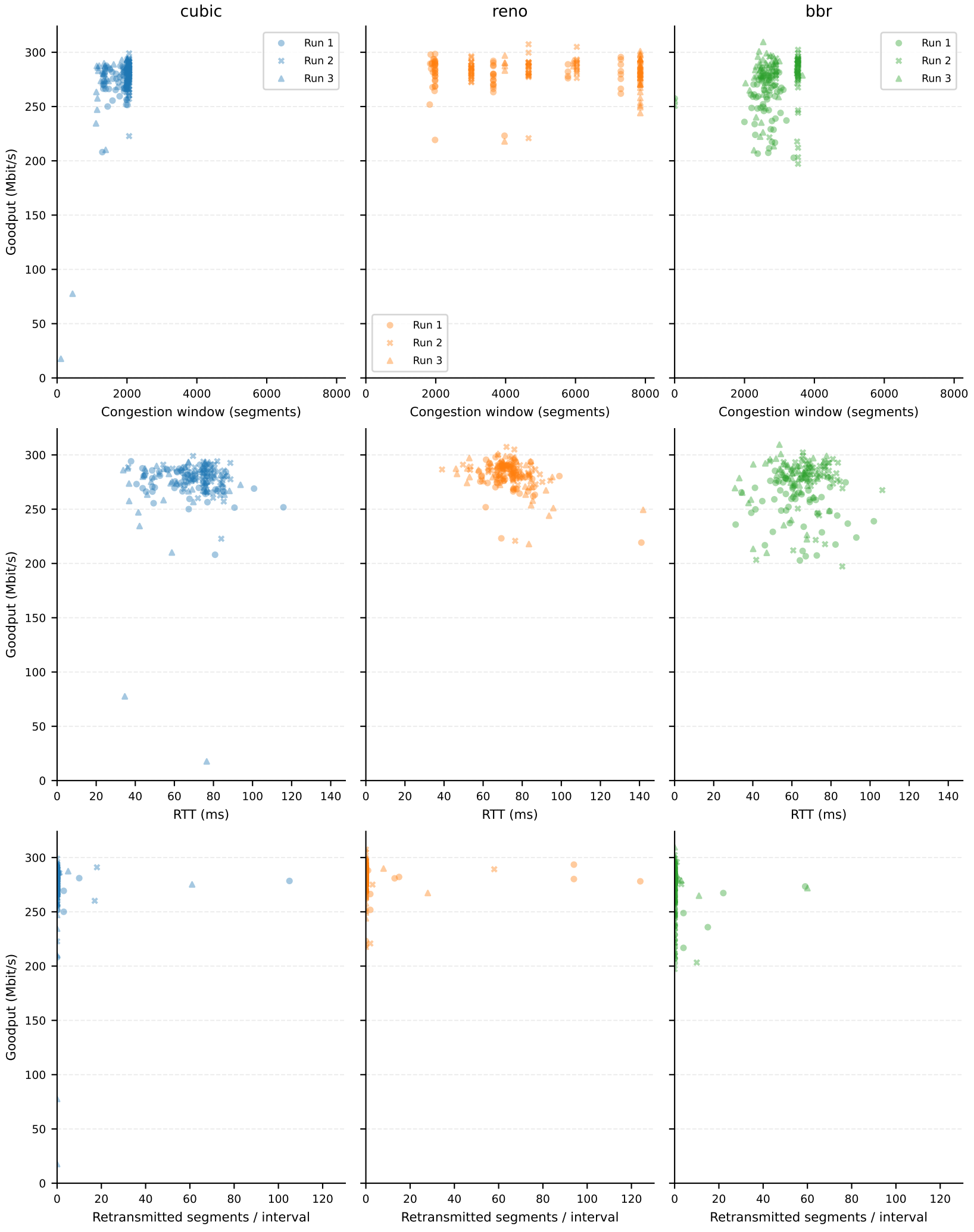
Public paths and traffic changed between tests. Three repetitions and one-second sampling give limited evidence, and small differences can reflect run variation. These single-flow experiments do not test fairness between competing flows.

[1] RFC 5681: Reno congestion avoidance

[2] RFC 9438: CUBIC

[3] Google BBR documentation


3(a): Representative destination — time series
cubic
reno
bbr
Run 1
Run 2
Run 3
Congestion window (segments)
RTT (ms)
Retransmitted segments / interval
Goodput (Mbit/s)
Time (s)


3(a): Representative destination — scatter plots
cubic
reno
bbr
Run 1
Run 2
Run 3
Goodput (Mbit/s)
Congestion window (segments)
RTT (ms)
Retransmitted segments / interval

3(b): Proposed congestion control algorithm

Our proposed controller uses RTT, goodput, and loss. Q3_DESIGN.md gives the full specification. It has not been implemented or evaluated; the experiments above compare the three existing algorithms.

Start with cwnd = 10 segments and an unlimited slow-start threshold, subject to kernel limits. Keep the minimum valid RTT since connecting and the previous eligible interval's goodput. Make one normal window decision per RTT with new ACK progress. Goodput is newly acknowledged bytes divided by actual elapsed seconds.

```
State: slow_start = true; previous_goodput = unset
On a new loss-recovery episode, once:
    request ssthresh = max(2, floor(cwnd / 2))
    let Linux apply recovery using that threshold
    slow_start = false; clear previous_goodput and interval accounting
On timeout:
    let Linux apply its normal timeout window and recovery
    slow_start = true; clear previous_goodput and interval accounting
Once per RTT with new ACKs, outside recovery:
    update minimum_RTT and measure goodput
    if app limited or receiver-window limited:
        clear previous_goodput; finish interval
    if slow_start and cwnd < ssthresh and RTT <= 1.10 * minimum_RTT:
        cwnd += min(newly_acked_segments, cwnd)
    else:
        slow_start = false
        if RTT > 1.25 * minimum_RTT and previous_goodput is valid
           and goodput <= previous_goodput:
            cwnd = max(2, floor(0.90 * cwnd)); ssthresh = cwnd
        else if RTT <= 1.10 * minimum_RTT:
            cwnd += 1
        else: hold cwnd
    obey kernel limits; previous_goodput = goodput; finish interval
```

Loss triggers a stronger response than delay. Low delay allows growth; high delay without better goodput causes a small reduction. This follows the window/rate mismatch at Dallas, retransmissions at Netfiber, and the IP-path stall. The 10% and 25% delay thresholds and reduction factors are initial choices to test. Linux handles timeout recovery without an extra controller reduction.

With equal RTTs and a shared bottleneck, additive growth and loss reductions may help flows share capacity. Different RTTs and competing loss-based algorithms may cause unequal rates. Repeated changes can produce oscillation, so fairness and convergence remain unproven. A connection-wide minimum RTT cannot adapt upward after a route change, and interval goodput can be noisy.

AI helped with code, plots, checking, and drafting. A teammate collected the Linux/WSL2 measurements.